



DBCLS BioHackathon 2026 report: Template for the very long title

First Author¹ and Last Author²

¹ First Affiliation ² ELIXIR Europe

BioHackathon series:

[DBCLS BioHackathon 2026](#)

Matsuyama, Japan, 2026

[data-engine-evaluation](#)

Submitted: 18 Sep 2026

License:

Authors retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC-BY](#)).

Published by [BioHackrXiv.org](#)

Introduction

As part of the DBCLS BioHackathon 2026, we here report our efforts to develop tools and workflows to extract target data from a triple store with complex schema structures. (italicized portion especially should be checked for correctness) The _GaSIC_INSERT DESCRIPTION OF GLYCOSMOS PORTAL. INSERT LARGE SCALE GOAL FOR CREATED A HUMAN SPECIFIC GLYCOBIOLOGY DATA SOURCE. AND WHY WE NEE

LLMs can navigate complexity in a highly organized and complex triple system quickly and accurately. Recognizing the advantages of this quality can improve efficiency of data transfer and data management between users and institutions. To that end, we developed a workflow for agent-assisted data extraction and a companion SPARQL endpoint tool with extended capabilities to facilitate cooperative work between subject matter experts to improve the supervision and validation of that agentic pipeline.

INSERT STUFF ABOUT HOW THIS LEADS INTO ENGINE DEVELOPMENT AND DATA WAREHOUSE MANAGEMENT HOUSEKEEPING

Meeting information

If you want to submit a preprint to BioHackrXiv, first check if your meeting is registered. You can find a list of meetings [here](#). If your meeting is missing, please contact your meeting organizers. The above list also provides information on the YAML fields with information about the meeting.

The following fields need to be given:

```
biohackathon_name: "DBCLS BioHackathon 2026"
biohackathon_url:  "https://2026.biohackathon.org/"
biohackathon_location: "Matsuyama, Japan, 2026"
group: YOUR-PROJECT-NAME-GOES-HERE
git_url: https://github.com/yourOrganization/your_report_repo
```

The [BioHackrXiv meeting pages](#) provide content to use for the first three fields. The `git_url` field must have the link to the GitHub repository with your preprint (draft).

Author information

Information about the authors is given in the [YAML](#) format at the top of this template. For authors you provide their names, their affiliations. That is the minimum, but as BioHackrXiv is moving to a situation where more metadata is shared, and used by, for example, EuropePMC, adding additional information is encouraged.

BioHackathons is about hacking together, and the minimal number of authors for reports is two. This makes a minimal example look like this:

```
authors:
  - name: First Author
    affiliation: 1
  - name: Last Author
    affiliation: 2
affiliations:
  - name: First Affiliation
    index: 1
  - name: ELIXIR Europe
    index: 2
```

Author identifiers

Ideally, authors provide their [ORCID](#) identifier. For affiliations, It is added with the `orcid:` field. So, an author record would look like this:

```
authors:
  - name: First Author
    affiliation: 1
    orcid: 0000-0000-0000-0000
```

Research Organization Registry identifiers

Matching the author identifier, the affiliations can be further specified with the [Research Organization Registry](#) (ROR) identifier. For example, this is the affiliation identifier can be added with the `ror:` field:

```
affiliations:
  - name: ELIXIR Europe
    ror: 044rwnt51
    index: 2
```

Contributor Role Taxonomy

A last feature since is minimal support for the Contributor Role Taxonomy (CRediT). You can specify the role of authors in writing the report with the `role:` field. However, the authors are responsible for selection the right terms from [CRediT](#). An example looks like this:

```
authors:
  - name: First Author
    affiliation: 1
    orcid: 0000-0000-0000-0000
    role: Conceptualization, Writing - review & editing
```

A full examples

A full example then has this structure:

```
authors:
  - name: First Author
    affiliation: 1
    role: Writing - original draft
  - name: Last Author
    orcid: 0000-0000-0000-0000
    affiliation: 2
    role: Conceptualization, Writing - review & editing
affiliations:
  - name: First Affiliation
```



```
index: 1
- name: ELIXIR Europe
  ror: 044rwnt51
index: 2
```

Formatting

This document use Markdown and you can look at [this tutorial](#).

Subsection level 2

Please keep sections to a maximum of only two levels.

Tables

Tables can be added in the following way, though alternatives are possible:

Table: Note that table caption is automatically numbered and should be given before the table itself.

```
| Header 1 | Header 2 |
| ----- | - - - - - |
| item 1  | item 2  |
| item 3  | item 4  |
```

This gives:

Table 1: Note that table caption is automatically numbered and should be given before the table itself.

Header 1	Header 2
item 1	item 2
item 3	item 4

Figures

A figure is added with:

```
![Caption for BioHackrXiv logo figure](./biohackrxiv.png)
```

This gives:



Figure 1: Caption for BioHackrXiv logo figure



Figures can be scaled by adding the width or height to the Markdown like this:

```
![Caption for BioHackrXiv logo figure](./biohackrxiv.png){ width=50px }
```

You can add cross references to figures by adding a LaTeX `\label{figureCode}` to the label of the Markdown figure and then use `\ref{figureCode}` to cite it:

```
![Caption for BioHackrXiv logo figure \label{figureCode}](./biohackrxiv.png){ width=50px }
```

This way, we can cite Figure 1.

Other main section on your manuscript level 1

Lists can be added with:

1. Item 1
2. Item 2

Citation Typing Ontology annotation

You can use [CiTO](#) annotations, as explained in [this BioHackathon Europe 2021 write up](#) and [this CiTO Pilot](#). Using this template, you can cite an article and indicate *why* you cite that article, for instance DisGeNET-RDF (Queralt-Rosinach et al., 2016).

The syntax in Markdown is as follows: a single intention annotation looks like `[@usesMethod In:Krewinkel2017]`; two or more intentions are separated with colons, like `[@extends:discusses:Nielsen2017Scholia]`. When you cite two different articles, you use this syntax: `[@citesAsDataSource:Ammar2022ETL; @citesAsDataSource:Arend2022BioHackEU22]`.

Possible CiTO typing annotation include:

- `citesAsDataSource`: when you point the reader to a source of data which may explain a claim
- `usesDataFrom`: when you reuse somehow (and elaborate on) the data in the cited entity
- `usesMethodIn`
- `citesAsAuthority`
- `citesAsEvidence`
- `citesAsPotentialSolution`
- `citesAsRecommendedReading`
- `citesAsRelated`
- `citesAsSourceDocument`
- `citesForInformation`
- `confirms`
- `documents`
- `providesDataFor`
- `obtainsSupportFrom`
- `discusses`
- `extends`
- `agreesWith`
- `disagreesWith`
- `updates`

There is a general `cites` intention, but this is already implied and should be left out. # Abstract The Human Glycome Atlas Project aims to integrate human glycoscience data into a linked RDF resource connecting glycans and glycoconjugates with their biological context. Constructing such a resource from existing knowledge bases requires reproducible identification and extraction of human-relevant data, preservation of semantic behavior when the resulting RDF is served through different graph engines, and appropriate control over access to protected

data. During DBCLS BioHackathon 2026, we developed an agent-assisted, human-validated workflow for exploring complex RDF schemas and generating SPARQL CONSTRUCT extraction queries. We further developed a cross-engine conformance framework for evaluating RDF serving behavior and advanced the AAll authorization architecture used to mediate access to TOHSA data. Together, these activities connect source-data discovery, reproducible extraction, semantic serving, and governed access into a common workflow for constructing and operating the TOHSA knowledge base.

Agentic Assisted Data Extraction

The workflow implemented at BioHackathon is an exchange of supervised task execution by the agent, and validation by the subject matter experts. The workflow (depicted below) consists of agent exploration, human validation, agent creation of extraction queries, human evaluation of those queries, and finally agent-facilitated querying of the endpoint with CONSTRUCT queries resulting in data files written in .ttl format. These files are meant to be compatible with mathematic evaluation via Exploratory Data Analysis (EDA) and/or semantic evaluation via human-facilitated querying. Developing the criteria for those evaluation steps was outside the scope of this BioHackathon, but it is planned as a future step for this project. All steps are intended to be iterable within and across themselves to allow for dynamic and documented changes resulting from discoveries about the data, changes in desired target or scope, or adjustment

INSERT extraction_workflow.png IMAGE

Multi-Step Schema Exploration

To provide understanding about the structure of the linked data, a three-step analysis is applied to each dataset. The agent determines which files in the context repository feed into which named graph, what predicate points to taxonomic information, and what corresponding object marks a resource as human. This is done while consulting the available config files, and ultimately confirming the veracity of all observations with live queries. The exploration queries return the full human IRI count, picking one richly connected, real instance and following every predicate out from it to determine range or relevant triples. This path exploration identifies what is safe to include in an extraction scope, what is a shared resource across graphs and potentially needs its own table, and whether or not a predicate carries a fan-out risk.

For each step, the agent creates an .md file that describes observations about the data structure. This analysis is not static, and if future steps reveal something previously unknown or poorly categorized, the agent returns to the .md document and amends it before the task is declared complete and the full .md file shared with the user. The details included in those observations are described below:

Step One, extraction criteria. Defines which predicate and value combination marks a resource as whatever the intended extraction target, in this case as human. This step runs queries with the intention of proving every claim with a live SPARQL query, and states plainly where a criterion rests in a dataset's documented scope. These queries are saved as separate files in the appropriate directory, and are directly hyperlink-referenced in the observations summary.

Step Two, entry counts. Runs count queries for target criterion from step one against the full live dataset, no LIMIT, and records the returned count with the query used to obtain it. Where more than one independent signal exists for the same criterion, this step check them against each other and investigates any disagreement with additional queries to the live endpoint rather than picking one arbitrarily.

Step Three, predicate discovery. Selects one richly linked, already confirmed instance and enumerates every predicate on every resource directly reachable from it, unrestricted. This is

where fan out hubs, a predicate that looks safe on one instance but explodes at scale because the target resource is itself cited by thousands of other records, opaque identifiers with no further content, and mislabeled predicates get caught, before they become a silent gap or an unbounded query in a production pipeline.

Below is a summarized example of the conclusions from an agent-assisted exploration of multiple datasets:

Dataset	Criterion	Count	Key finding
Disease	Two levels, DOID membership (concept) plus gene or glycoprotein glycan:has_taxon (molecular evidence)	4115 of 4372	The invented pipeline:hasPhenotype predicate was found and replaced with the real four hop bridge
Pathways	biopax3:organism, direct, confirmed two independent ways	2870 of 23486	Three pipeline bugs found in a deep cross check, a self referencing related pathway bug, a missed nested sub pathway reaction gap, and an uncollected output variable
Genes	glycan:has_taxon directly on glycan:Glycogene	10276	A ggdb (GlycoGene-DataBase) sub analysis was built separately on that named graph's own rich reaction content
Glycoproteins	glycan:has_taxon directly on glycan:Glycoprotein	16711	A numeric pattern gene target's own triples span six graphs, not two; a related_graphs sub analysis covered four further graphs (gpdb, HPA, lipidmaps_gene, protein_egf)
Lectins	glycan:has_taxon directly on sugarbind:Lectin	318 of 6298	CarboGrove, reached through one rdfrs:seeAlso hop, expands into 1.3 million triples, a confirmed fan out hub, excluded

Dataset	Criterion	Count	Key finding
Glycans	Two hop glycan:is_from_source/glycan:has_taxon	8042 of 265401	A directory named external holds none of the resource type its name implies; the extraction package's own inference dependent query proved unnecessary; two pipeline bugs found and fixed
Glycolipids	None exists, confirmed by exhaustive live checking	6046 (full population, no species filter possible) of 6280 store wide	No species predicate anywhere in the dataset; the scope decision to cover the full population was made without a response from the person directing the work, and is flagged for confirmation

CONSTRUCT Query Drafting

Once the subject matter confirms that the agent has an accurate representation of the data structure and how the target data fits into that structure, the agent begins to draft construct queries. Those queries are crafted by.....

INSERT INFORMATION FROM AGENT

The user evaluates the query for a) appropriate scope b) time-out risk and c) extraction coverage completeness. Despite being more computationally intensive than SELECT files, CONSTRUCT queries do not require an additional step of triple reconstruction across tables, and ensure fidelity between target triples and the extracted triples. All drafted CONSTRUCT queries are documented before being run against the endpoint, allowing for a subject matter expert to run the query with LIMIT conditions and evaluate the output.

FUTURE CAPABILITIES The documentation created by this workflow is prose heavy and could be simplified into a more structured format that is still human readable, but easier for other agents to engage with. Standard RDF-config files in .yaml format are already commonly used to document RDF schema, and could serve as an appropriate addition to the prose. The simplified and structured .yaml files could facilitate new ways to extend the workflow beyond extraction as we continue development.

Ontological Data Investigation Nexus (ODIN)

INSERT LABELED SCREENSHOT HERE

...

Acknowledgements

...

References

Author contributions

First Author: Writing – original draft; Last Author: Conceptualization, Writing – review & editing.

References

Queralt-Rosinach, N., Pinero, J., Bravo, À., Sanz, F., & Furlong, L. I. (2016). DisGeNET-RDF: harnessing the innovative power of the Semantic Web to explore the genetic basis of diseases. *Bioinformatics*, 32(14), 2236–2238. <https://doi.org/10.1093/bioinformatics/btw214> [cito:citesAsAuthority].



Appendices

If you want the Appendix (-ces) to show up after the references, wrap them in after the header, like done in this Markdown file. Look at the [source](#) to see the exact structure.